Tel Aviv University

The Iby and Aladar Fleischman Faculty of Engineering

FPGA Laboratory

Experiment FPGA 4 – Dual Stopwatch

Written by: Leo Segre and Osher Stamker

Based on Konstantin Berestizshevsky's work

Lab Goals (This is also the material for the test)

The purpose of this lab is to cover the following topics:

1. Basic design and simulation in Verilog
    a. Module description and instantiation, parameterized modules
    b. Design and implement FSM and write it into a Verilog code
2. FPGA Design Flow in Xilinx Environment:
    a. Synthesis, Implementation and the constraints file (.xdc)
    b. Timing analysis (setup, hold, slack)
    c. Interfacing the peripheral components of the BASYS3 board.
    d. FPGA device programming
3. Electronic issues
    a. Digital debouncing (use hysteresis counters) and pulse generation
    b. Clock, frequency division

Submission Guidelines

The lab consists of 15 tasks, where each task depends on the previous.

Task 14 is optional and only needed in case you are having trouble debugging your design after synthesis. Parts of task 15 are also marked as optional. Only if you did task 14 you have to do those parts of task 15 as well.

All tasks must be submitted in your progress report. At each submission deadline, add the current state of the tasks to your previously submitted report.

1. A draft for the progress report with additional instructions is available in Moodle.
2. Submit the progress report (PDF format) and all your *.v files. All zipped together in a zip file named FPGA4 <ID1> <ID2>
3. Each waveform must be annotated by (i) drawing arrows that point to important signals and (ii) explaining these signals.
4. Most of the modules in this lab are parametric (for example, Compadder(N) has a design parameter N, which determines the bit-width of its operands). These modules must in fact support different parameter values, and not to stick to some constant value.
5. Each Verilog source/test-bench header must have both of your names in it:

```
1  `timescale 1ns/10ps
2  //////////////////////////////////////////////////////////////////
3  // Company:         Tel Aviv University
4  // Engineer:        Your name
```

1. Create new Vivado project

- At home you can work with the Vivado Web-Pack. You will need to create a free student account in Xilinx before downloading and installing the software.

- In the lab, you should save your project in in **G:\FPGA_LAB.** It is your responsibility to either use a USB-flash drive or other cloud solutions to save your project before you leave the laboratory. The laboratory PCs are erased from time to time.

Launch the Vivado software, create a new project. Follow the "Vivado Project Creation" subsection of the Vivado-tutorial to set up a new project. The name of the project should be **lab4**, and it should be saved in a **path that doesn't contain non-ascii characters and doesn't contain spaces**. The project type is **RTL Project**. You may add the provided Verilog sources as well as lab1_constraints.xdc constraints-file. Make sure to choose the **xc7a35tlcpg236-2L** part in the "Default Part" screen of the wizard. Notes:

- To add more sources later, go to "Project Manager Add Sources" in the flow-navigator in the left part of the Vivado screen.
- For this project you will need to use sources from previous designs: FA.v, Compadder.v, Lim_Inc.v, Counter.v, Seg_7_Display.v
- In this lab, all the tasks are to be performed under the same "lab4" project.

2. Control

Consider the synchronous module Ctl

```
module Ctl(clk, reset, trig, split, init_regs, count_enabled);
   input clk, reset, trig, split;
   output init_regs, count_enabled;
```

Functionality is described by the FSM in Figure 1.

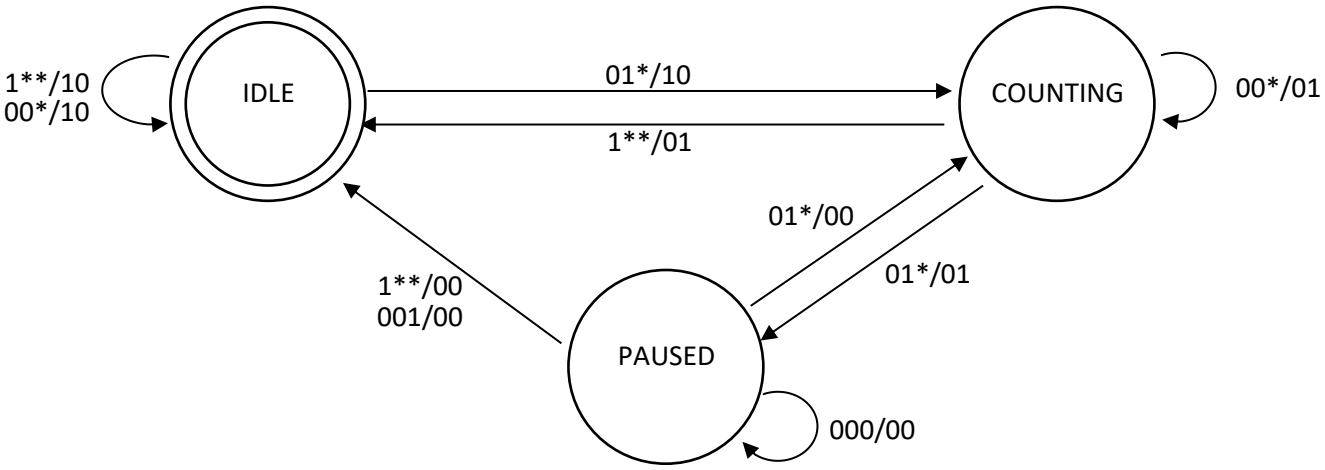


*Figure SEQ Figure * ARABIC 1 – The FSM of the stopwatch control. The transition captions have the following format: reset(t),trig(t),split(t)/init_regs(t),count_enabled(t). A star (*) stands for "don't care" value.*

a. Add the provided design source Ctl.v and the provided simulation source Ctl_tb.v to your project.
b. Design a synchronous module Ctl by filling the Ctl.v Verilog source.
c. Write a test-bench for your design by filling the provided Ctl_tb.v source, perform a behavioral simulation and make sure to receive a "Test Passed" indication in the TCL console. Note: a sufficient test would stimulate transitions to all possible states and compare the outputs against the expected ones.
d. **In the progress report:**
    i. Provide an annotated waveform of your simulation. In addition, take a screenshot of the Tcl console window showing "Test Passed" message and the "$finish called…" message. Explain what is shown in the simulation.
    ii. The FSM is presented as a Mealey state diagram. Exlpain whether this FSM is a Moore FSM.

3. Debouncer

Consider the Debouncer(COUNTER_BITS) module:

```
module Debouncer(clk, input_unstable, output_stable);
   parameter COUNTER_BITS = 7;
   input clk, input_unstable;
   output reg output_stable;
```

Your aim is to implement the debouncing of the input signal using a counter approach. Namely, maintain a saturated counter[1] and increment it every clock cycle when unstable input equals 1 and decrement the counter every clock cycle when unstable input equals 0.

a. Add the provided design source Debouncer.v to your project.
b. Design the Debouncer(COUNTER_BITS) by filling the Debouncer.v code. Your debouncer should generate a **single-cycle pulse** as a stable output. Namely, the stable output will rise only for a single clock cycle and then go low until the next time the user asserts an (unstable) input value 1.
c. **In the progress report:** Discuss the tradeoff between choosing high and low values for the counter, and how is this related to the low-pass-filter debouncing approach we saw in the previous experiments.

[1] An N-bit saturated counter can be incremented to the maximum value of 2^N-1 or decremented to a minimum value of 0, but cannot perform "wrap-around", namely: 2^N-1 **+1 = 2^n-1** and 0-**1=0**

4. Stopwatch

The Stopwatch module is a top-level module of our design, which interfaces the IO blocks of the FPGA in order to receive button signals and to drive the 7-segment display. The Stopwatch's functionality must provide two independent stopwatches (one displayed on the 2 left hand side digits of seg-7 display, and another stopwatch is displayed on the 2 right hand side digits thereof).

How to associate the inputs to Stopwatch module with its internal modules: The central user button will serve as a reset for the entire system, the upper button as a trigger, the right button as a split, the left button will toggle the selection between the left and the right stopwatches. The LEDs underneath the 7-seg component should indicate which of the two stopwatches is currently selected. All the user buttons should undergo a debouncing.

```
module Stopwatch(clk, btnC, btnU, btnR, btnL, seg, an, dp,
led_left, led_right);
    input              clk, btnC, btnU, btnR, btnL;
    output  wire [6:0] seg;
    output  wire [3:0] an;
    output  wire       dp, [2:0] led_left, [2:0] led_right;
```

a. Design a block diagram for the Stopwatch module, use the previous designs as an instantiated modules (No need to show the internal logic), show and explain any other logic and connectivity.
b. Add the provided design source Stopwatch.v to your project.
c. Use the modules you constructed so far and fill in the code of Stopwatch.v, This is the place where you should design a certain circuitry that will utilize two Ctl and two Counter models and toggle the control between them using btnL.
d. Set Stopwatch as a Top module under the "Design Sources" folder.

5. Elaborated Design

a. In the flow navigator, RTL analysis, open the elaborated design schematic, make sure that the schematic seems correct and no inputs or outputs are missing.
b. **In the progress report:** Provide the screenshot of the elaborated design schematic.

6. Synthesis

a. Synthesize the design (In the flow navigator, go to Synthesis Run Synthesis).

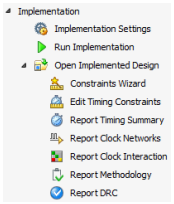
b. Read the “Messages” tab (near Tcl Console, in the bottom of Vivado window) and **fix your design until no errors, critical warnings or warnings remain.**
c. **In the progress report:**
    i. Provide a screenshot of the synthesized design schematic.
    ii. Describe the differences that appeared in the synthesized design compared to the elaborated design.
    iii. Describe the issues you encountered during synthesis and how you solved them (errors, critical warnings, and warnings).

7. Setting up constraints

The constraints file defines the clock pin, clock constraints, and the IO your design will interface the board with.

a. Open the lab4_constraints.xdc file, make sure it is added to your project.
b. Set up the clock signal input by uncommenting the 2 lines that connect the clk signal of your top module to the W5 pin of the FPGA.
c. Provide a timing constraint by uncommenting the line that begins with create_clock command
d. Set up the input IO pads by uncommenting the lines that interface the btnR, btnC, btnU and btnL inputs of your design.
e. Set up the output IO pads by uncommenting the lines that interface the "seg" and the "an" buses, as well as the "dp" output line (the output that is in charge of the dot nearby each one of the 7-seg digits). In addition, pick six led output pins (lines 65-76 of the constraints file) and map them to led_left[2:0] and led_right[2:0] ports of the Stopwatch.

8. Implementing the design



a. In the flow navigator, go to Implementation Run Implementation
b. After implementation is complete, check (again) for errors and critical warnings.
c. **In your progress report:**
    i. Provide a screenshot of the design timing summary where you must exhibit positive slack both for setup and for hold times.
    ii. Specify the lengths (in nanoseconds) of your critical setup path (i.e. *path length* with the lowest setup slack, do not just specify the slack value).

9. FPGA programming and debugging

Warning: Do not proceed to FPGA programming if the implementation procedure ended with critical warnings or errors as it might damage the FPGA device!

In this part you will work with the final stage of Vivado's Flow Navigator, called "Program and Debug"

a. Follow the three steps from the "Programming the FPGA" section in the Vivado-tutorial (1-bitstream, 2-connect, 3-program via Jtag).

Pay attention to the correct orientation of the USB cable as shown:

b. You need to make sure you see a correct functioning of your design. In the final meeting at the lab, you will need to show an instructor the working design. Make sure to prepare it at home in advance. You won't have time to fix the design at the lab.

c. After your design was successfully approved, **remove the debug core** (see Vivado-tutorial document), re-implement the design. Provide a screenshot of your Project Manager Project Summary screen including post-implementation Timing, Power, Utilization details (table view).